Namma Space: Smartphone Video to a Walkable Photorealistic Indoor Twin

Team ID: TM-12354C3F6F2 · **Institute:** Vellore Institute of Technology · **Round 1 Technical Abstract**, Techfest 2026-27, IIT Bombay

Members: Nilesh Kumar Srivastava, Ojas Ashish Kulkarni, Ujesha Sivaramakrishnan, Ashwin Dandotiya

All links, including the live viewer, repository, dataset and demo video, are in *Additional_Links_and_Prototypes.pdf*.

Summary. Namma Space turns a few minutes of ordinary phone video into a photorealistic 3D copy of an indoor space that anyone can walk through in a web browser. One person films each room by following a short capture procedure. Our pipeline then extracts sharp frames, recovers the camera path with COLMAP, trains a 3D Gaussian Splatting model with Brush and compresses it to a web-sized file. The result streams from a content delivery network into a lightweight viewer built on three.js and Spark, where visitors move with the keyboard and mouse or by touch on a phone. Every stage checks its own output before the next one starts, and a person walks the finished model before we publish it.

1. Problem and scope

Imagine arriving at a large store, an exhibition hall or a festival venue for the first time. A floor plan tells you where the rooms are, but not what they look like, where the entrance you want actually is, or whether the stall you are looking for is behind that pillar. Photo tours show what a place looks like but cannot be explored freely. What a visitor really wants is to walk the space before they arrive.

Our answer is a photorealistic digital twin that a non-expert can create with a phone and that anyone can explore in a browser with nothing to install. Round 1 delivers this core loop: capture a space, reconstruct it faithfully and roam it freely. We designed the capture procedure and the automated checks so that quality comes from following the process, not from manual 3D modelling. The repository already contains groundwork for the later rounds. A separate studio app holds a scene format with points of interest, search, and route planning over a walkable grid, which Round 2 will attach to the Round 1 model. Round 3 adds on-site capture by pairing a phone with the studio, plus optional AR navigation.

2. System architecture

We split the system into an offline reconstruction pipeline and a purely static web side. Reconstruction needs a GPU and takes a while, so it runs once per capture on a single workstation. Viewing must be instant and work on any device, so the web side has no server code at all: the app and the model are plain files served from fast, cache-friendly hosts. This separation lets us update the model without redeploying the app, and the reverse.

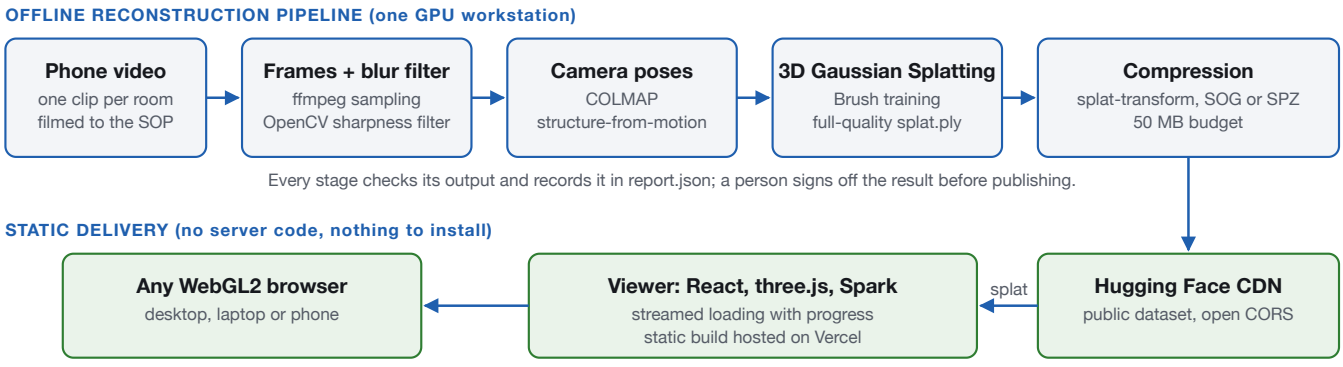


Figure 1. End-to-end architecture. Heavy work runs offline on one workstation; the web side is two static origins, the app on Vercel and the model on a CORS-enabled CDN.

The pipeline is one Python command-line tool with resumable stages. Each run keeps a single report file that records stage timings, every check result and the run's status, so anyone can see exactly how a model was produced. The viewer reads a small scene file holding the model's address, starting position, scale and orientation, so publishing a new capture never requires a code change.

Hardware. Only one step needs an NVIDIA GPU: COLMAP's feature extraction and matching, for which we use its CUDA build. COLMAP can also run these steps on the CPU, more slowly. Brush trains through WebGPU, so the training step runs on any recent GPU from NVIDIA, AMD, Intel or Apple. Viewing needs only a browser with WebGL2.

3. Capture procedure (SOP)

Reconstruction quality is decided at capture time. A blurred or badly lit video cannot be repaired later, so we wrote the procedure around the assumptions the algorithms make: a single unchanging camera, a static scene, and every surface seen many times from different positions. Each rule below exists to protect one of those assumptions.

Rule	Why it matters
Fix the camera: lock exposure and focus, use the 1x lens without zoom, turn stabilisation and HDR off, and change nothing mid-clip	Structure-from-motion models one camera with fixed optics. Zoom, digital stabilisation and focus hunting quietly change that camera and bend straight walls. Changing brightness confuses feature matching and bakes flicker into the model's colours.
Move slowly: walk at about 0.5 m/s, pan no faster than about 20° per second, never spin in place	Motion blur is the most common reason a reconstruction fails. At our sampling rate of 3 frames per second, this pace puts consecutive frames about 17 cm apart, which gives dense overlap with little blur.
Cover everything twice: walk the perimeter facing slightly into the wall, make two passes through the middle, pan slowly at corners, arc around large furniture, cross doorways slowly in both directions and finish where you started	Each surface needs to appear in at least three frames from different positions to be placed accurately in 3D and coloured correctly from every angle. Returning to the start closes the loop so accumulated drift can be corrected, and two-way doorway crossings keep adjoining rooms connected.
Light it evenly: all lights on, curtains open, no direct sun beams and no flash	Consistent lighting gives consistent colour across views. Blown highlights and moving flash shadows end up permanently painted into the model.
Avoid what cannot be reconstructed: mirrors, glass, windows, plain white walls and people walking through	Reflections, transparency and movement break the static-scene assumption and leave floating artefacts. Blank walls give the feature matcher nothing to hold on to, so we keep edges, switches or posters in frame.
Film long enough: at least 31 seconds per room, ideally 60 to 120, in landscape at 1080p or 4K and 30 fps	We need at least 80 sharp frames. At 3 frames per second, 31 seconds yields 93 frames, and discarding the blurriest 15 % leaves exactly 80. The pipeline applies this rule to the clip's real length before doing any work and tells the user the minimum.

Before leaving the site, the person filming spends a minute scrubbing through the video for blurred or dark sections and checks that the loop is closed and every doorway was crossed both ways. The SOP also lists common failure modes with their fixes, and explains how to set real-world scale from one known length such as a doorway.

4. Pipeline design

We built the pipeline to fail early and loudly rather than produce a poor model quietly. Each stage checks its own output, and a run moves forward only when those checks pass. When a stage stops, its message tells the operator what to change, usually in the capture rather than in the settings. Finished stages are kept, so a failed run resumes from the point where it stopped.

Validation

Before any processing, the pipeline confirms that every tool is installed and that the video can produce a usable model: high enough resolution, and long enough to yield 80 sharp frames at the chosen sampling rate. Running the same check that extraction later relies on means a clip that passes validation will not fail for lack of frames twenty minutes later.

Frame extraction

We sample the video at 3 frames per second, which matches the walking pace in the SOP, and scale frames to at most 1,600 pixels on the long side. OpenCV then scores each frame's sharpness using the variance of its Laplacian,

and we discard the blurriest 15 %. Removing a fixed share adapts to each video's overall sharpness, so we need no hand-tuned threshold.

Camera poses
COLMAP finds SIFT features in every frame, matches them between neighbouring frames, which suits video walked in order, and solves for every camera's position along with a sparse point cloud. It treats the whole clip as one camera with fixed optics, which is what the capture rules guarantee. We accept the result only if at least 60 % of frames are placed and the point cloud is dense enough. We flag fewer than 90 % placed frames, a high reprojection error or a room that splits into separate pieces, and our acceptance bar for publishing is 90 %. The frames are then undistorted for training.

Splat training
Brush trains a 3D Gaussian Splatting model for 30,000 steps and writes one full-quality file at the end. Before training, we confirm the dataset contains exactly one camera solution and every image COLMAP placed. Brush would otherwise train silently on fewer views, or start from a leftover model file in the folder. A trained model is reused only if the trainer, the step count and the camera solution are all unchanged, so a changed setting always produces a fresh model. Afterwards, we verify the file has every attribute a splat needs, is not truncated, contains no invalid values and has at least 50,000 splats.

Compression
A raw splat of a room can run to hundreds of megabytes, far too much for a phone on mobile data. We convert it with splat-transform to the SOG format against a 50 MB budget. On our synthetic test room, SOG was about ten times smaller than the raw file. If the file is still too large, we first remove the higher-order colour detail that models subtle view-dependent shading, and only then thin out splats evenly. If SOG encoding fails on a machine, we retry on the CPU and finally fall back to the SPZ format, so this stage does not depend on one encoder. The full-quality file stays on the workstation as the master copy.

Publishing and sign-off
The pipeline prepares the compressed model and its scene file for the local viewer, where the operator aligns scale, orientation and starting point and then walks the whole room. Automated checks prove a file is well formed. Whether it looks right is a human judgement, so a run is marked successful only when that person approves it with a short note. Our bar is at least 90 % of frames placed, no floating blobs or smeared walls along the walking path, and recognisable doorways and furniture.

5. Web viewer

We render Gaussian splats rather than a textured mesh because splats reproduce what makes a room look real: soft lighting, glossy surfaces that change with viewpoint, and thin or cluttered objects that photogrammetry meshes tend to blur or break. They also need no dense multi-view reconstruction, meshing or texture baking. The trade-off is that a splat has no clean surface for collision, which we handle with a separate navigation layer (Section 7).

The viewer is a React app using react-three-fiber over three.js, with Spark drawing the splats inside an ordinary three.js scene. Keeping a standard scene graph means points of interest, route lines and collision helpers in later rounds are simply more objects in the same scene.

Controls. On a computer, the visitor clicks once to enter the space, looks around with the mouse and walks with W, A, S and D or the arrow keys. Shift is a sprint, Q and E lower or raise the eye height, and Escape releases the mouse. Movement stays level with the floor wherever the visitor looks, and an optional boundary keeps them inside the room. On a phone or tablet, dragging one finger looks around and holding two fingers walks forward.

Loading. The model streams in with a progress bar that shows megabytes received and then a short decoding step, so a visitor on a slow connection always sees that something is happening. If the model cannot be fetched, the viewer explains why in plain language, for example that the file is missing, private or blocked by the host, instead of showing a blank screen.

Hosting. The studio app and viewer are a static build on Vercel, live at [namma-space-studio.vercel.app](), of about 3.7 MB, whose hashed files browsers cache indefinitely. The splat model lives in a public Hugging Face dataset, whose CDN allows cross-origin requests from any site. Because the app and the model are hosted separately, publishing a new capture is a file upload, not a deployment.

6. Results

Round 1 demonstrates the full software pipeline on a synthetic fixture scene. A real-capture attempt (70 WhatsApp-compressed photos of a room) registered 57 % of frames — below our 60 % acceptance bar — because of insufficient overlap at doorway transitions and low-resolution source images. The pipeline correctly refused to proceed, validating that the quality gates work as designed. The table below reports the software verification results.

Metric	Value	Metric	Value
Unit tests (Vitest)	679 / 679 passed (36 files)	Python tests (pytest)	78 / 78 passed
Browser E2E (Playwright)	35 specs: all pass in isolation; 33 pass, 2 flaky, 1 timeout in one software-WebGL run	Scene validation	2 / 2 bundles PASS
TypeScript typecheck	0 errors	ESLint	0 errors
Production build (gzip)	142 KB entry, 523 KB viewer (lazy)	Synthetic splat fixture	~106,000 splats, SOG format

Capture Studio: multi-image upload, room/view-group organisation, ZIP export with integrity hashes

Figure 2. The Capture Studio workspace with image thumbnails, room rail, and export controls.

Spatial Viewer: synthetic fixture with WASD navigation, POI search, A routing, and scene selector*

Figure 3. First-person walkthrough of the synthetic fixture scene with HUD overlay.

The synthetic fixture validates the entire viewer pipeline. Real-capture reconstruction requires a re-shoot with native-resolution video following the documented SOP (see docs/CAPTURE_DAY.md).

7. Limitations and next steps

Collision and navigation. In Round 1 the visitor is kept inside an optional rectangle, so walls and furniture can be walked through. For Round 2 we will add a walkable grid in real-world metres, already prototyped in the studio app, and fit the floor plane to COLMAP's point cloud so that alignment no longer needs a manual step.

Points of interest and routing. The studio app already defines a validated scene bundle that combines a navigation grid, points of interest and a quality report, together with point-of-interest search and shortest-path routing on the grid. Round 2 connects these to the Round 1 model in a shared coordinate frame.

Reconstruction limits. Mirrors, glass and blank walls remain difficult, and scale and orientation are currently set by hand from one known length. Compression is lossy, so we compare each published model against its full-quality master before release.

Scaling to larger venues (plan). Our design for a building-sized venue such as IIT Bombay's Lecture Hall Complex is to treat each room or corridor segment as its own capture. Each one follows the same SOP and is processed independently, so captures can be spread across several workstations. Doorway crossings filmed in both directions give neighbouring captures shared views, which we would use to register them into one coordinate frame. At our estimated 30 to 50 minutes of processing per room, a floor of about twenty spaces needs roughly 10 to 17 GPU-hours, which several machines can complete overnight. For a space that size we would switch COLMAP from sequential matching to vocabulary-tree matching so revisited areas are recognised, and stream the model as spatially tiled, level-of-detail chunks so the browser loads only the area near the visitor. None of this is built yet. Round 1 targets one room at a time.

Round 3. On-site capture by pairing a phone with the studio, building on the pairing and frame-transfer prototypes already in the repository, and optional AR navigation against the twin.

8. Open-source credits

Tool and version	Use in Namma Space	Licence
COLMAP 4.2.1	feature extraction, matching, structure-from-motion, undistortion	BSD-3-Clause
Brush 0.3.0	3D Gaussian Splatting training	Apache-2.0
FFmpeg and ffprobe	frame sampling, video inspection	LGPL-2.1+ (GPL-2.0+ for GPL builds)
OpenCV (opencv-python-headless)	frame sharpness scoring, image checks	Apache-2.0
NumPy	splat file validation	BSD-3-Clause
@playcanvas/splat-transform 3.10.3	conversion to SOG and SPZ, colour-detail reduction, decimation	MIT
Spark (@sparkjsdev/spark) 2.4	Gaussian splat rendering in three.js	MIT
three.js 0.186	WebGL scene and camera	MIT
react-three-fiber 9	React bindings for three.js	MIT
React 19 and React DOM	viewer and studio interface	MIT
Vite 7 and 8, TypeScript 5.9 and 6, oxlint, ESLint	build, type checking, linting	MIT (TypeScript: Apache-2.0)
Node.js 22+	build tooling, runs splat-transform	MIT
PlayCanvas engine 2	studio viewer prototype for Round 2	MIT
zod 4, fflate, idb, ws, qrcode-generator	studio schemas, bundle export, browser storage, development pairing relay, QR codes	MIT (idb: ISC)
Vitest 5, Playwright	studio unit and browser tests	MIT (Playwright: Apache-2.0)
Archivo and IBM Plex Mono	studio typography	SIL OFL-1.1
GSAP 3	studio interface animation	GSAP Standard no-charge licence (not OSI-approved)

Hosting services, not open source: Vercel for the web app and Hugging Face Hub for model storage and delivery.